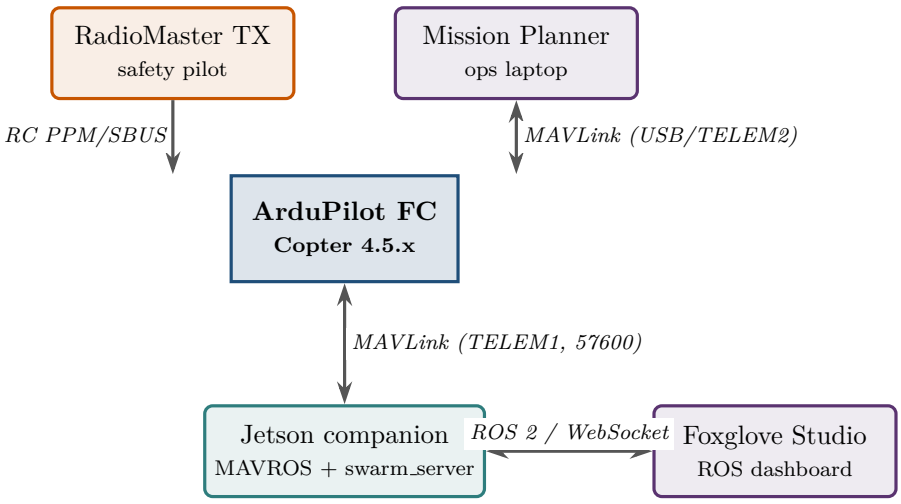
# Orynth v2

End-to-End System: from Bench to Swarm




**The single idea that makes the rest of this document make sense:**

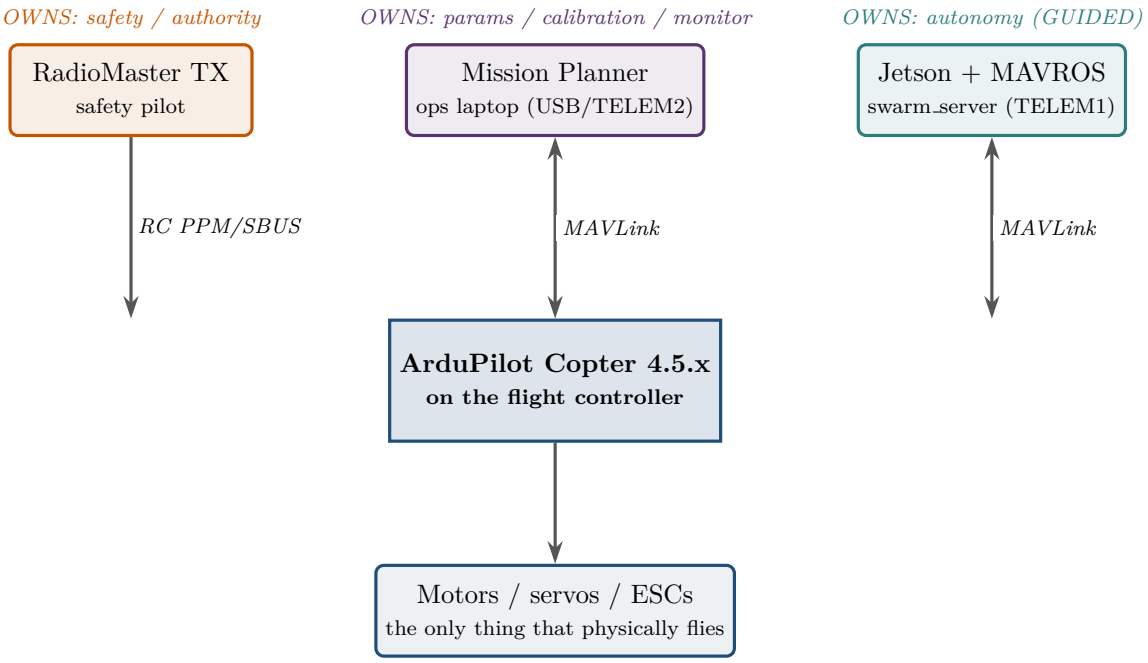
The **ArduPilot firmware** on the flight controller is the only thing actually flying the drone. Mission Planner, your RadioMaster TX, and the Jetson companion are all *clients* talking to that firmware over MAVLink or RC. This document shows how those clients connect, what each does, and how the same architecture scales from one drone on a bench to five drones in formation.

Reference document — diagrams compile from source; photos are placeholders to be filled.
Rebuild: `pdflatex end_to_end_setup.tex` (twice for cross-refs).
May 22, 2026

# Contents

## 1. The big picture: three channels, one firmware

ArduPilot Copter is a multi-client autopilot. Three independent input channels can address the same firmware simultaneously without conflict; each has a fixed role, and the firmware enforces a strict priority when they disagree (§10).

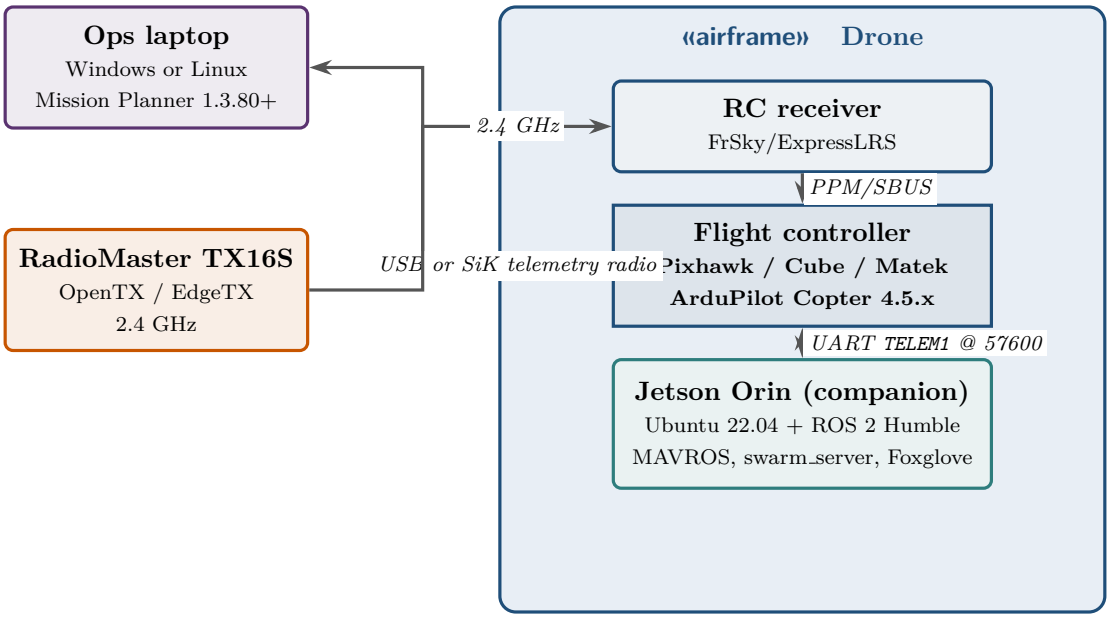



**What this means for your current bench setup**

You already have the orange and purple channels. The Jetson + MAVROS layer is *added in parallel*, on a different serial port (`TELEM1`). Nothing about your current RadioMaster + Mission Planner flow changes. RC override always wins; you can always take a drone manually at any instant.

## 2. Hardware: a single drone on the bench

UML deployment view of one drone as it will be wired for Phase 2.5b. The flight controller is the central node; everything else attaches to it over a well-defined physical link.



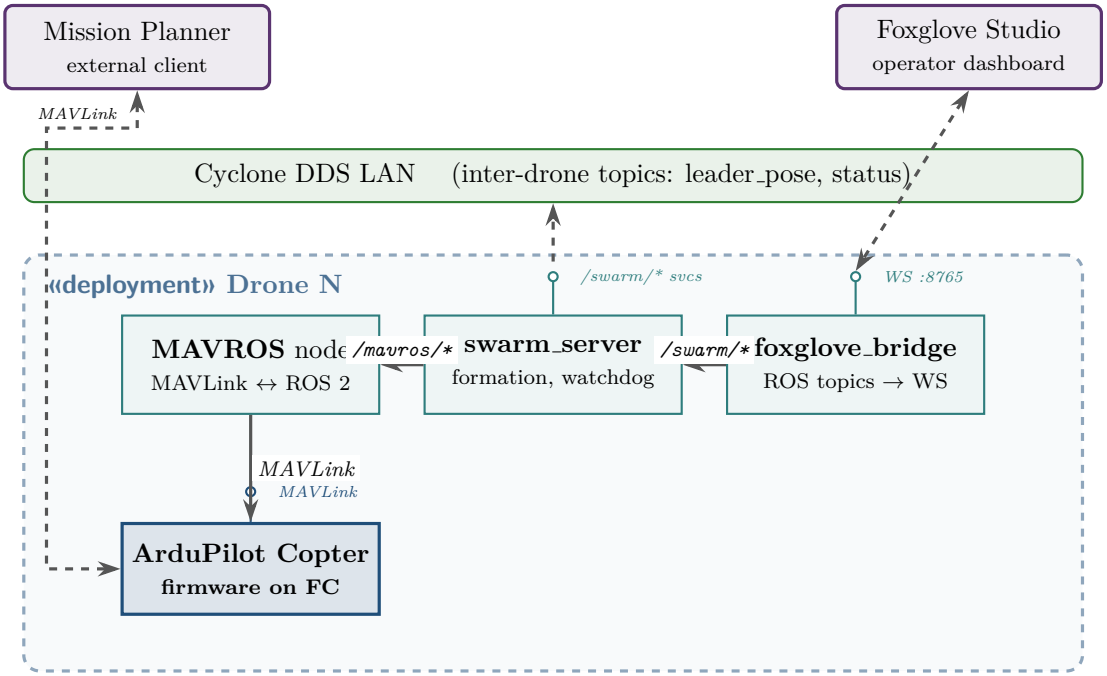

| Link | Port | Baud / type | Carries |
|---|---|---|---|
| Mission Planner ↔ FC | USB or `TELEM2` | 57600–921600 | MAVLink: params, mode, mission, monitoring |
| RadioMaster TX → RX → FC | RC input pins | PPM or SBUS | Stick channels, mode switch (LOITER/POSHOLD/GUIDED) |
| Jetson ↔ FC | `TELEM1` UART | 57600 | MAVLink: state, setpoints, arming, missions |
| Jetson ↔ Operator | WiFi (Foxglove WS `:8765`) | TCP/IP | ROS 2 telemetry, services, status |

*[ photo placeholder ]*

Suggested shot: the airframe on a stand with the FC visible, the RC receiver taped/zip-tied, the Jetson cabled into `TELEM1`, and the USB cable running to a laptop with Mission Planner open in the background. Drop the JPG into `images/bench_single_drone.jpg` and uncomment the \includegraphics line in the source to replace this placeholder.

## 3. Software on the companion: UML component view

What runs on the Jetson, and how the pieces talk. UML component diagram with "provided" (lollipop) and "required" (socket) interfaces.




**MAVROS**
One process per drone. Translates ArduPilot MAVLink to ROS 2 topics (`/drone_N/mavros/state`, `/.../local_position/pose`, etc.) and exposes services to arm, set mode, take off, send setpoints. The only thing in the stack that speaks MAVLink directly.

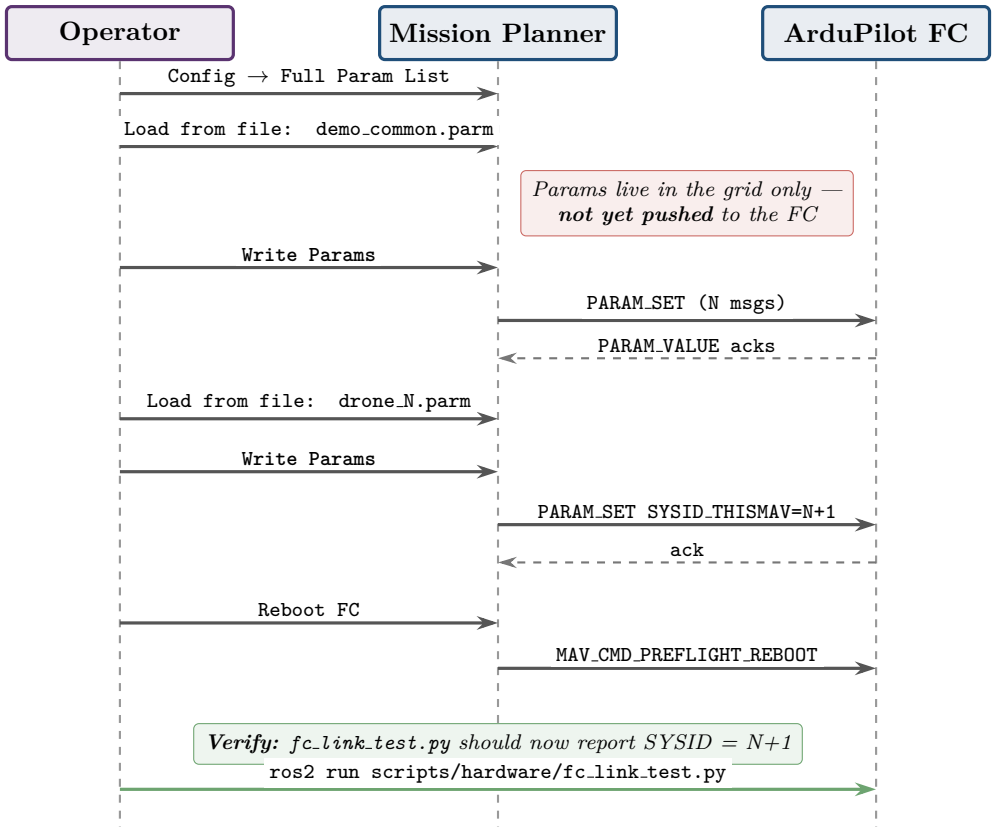
**swarm_server**
Owns the formation logic and the leader-pose watchdog. Exposes `/swarm/takeoff`, `/swarm/follow_leader`, `/swarm/land`, and a `/swarm/status` stream. Drives followers by publishing setpoints to MAVROS.

**foxglove_bridge**
Plain ROS 2 → WebSocket exporter. Lets Foxglove Studio (on the ops laptop) subscribe to any topic over `ws://`. No control authority.

**Cyclone DDS**
Underlies all ROS 2 traffic on the swarm LAN; *leader_pose* and shared status topics are the inter-drone fabric (ADR 0004).

## 4. Sequence: loading airframe parameters with Mission Planner

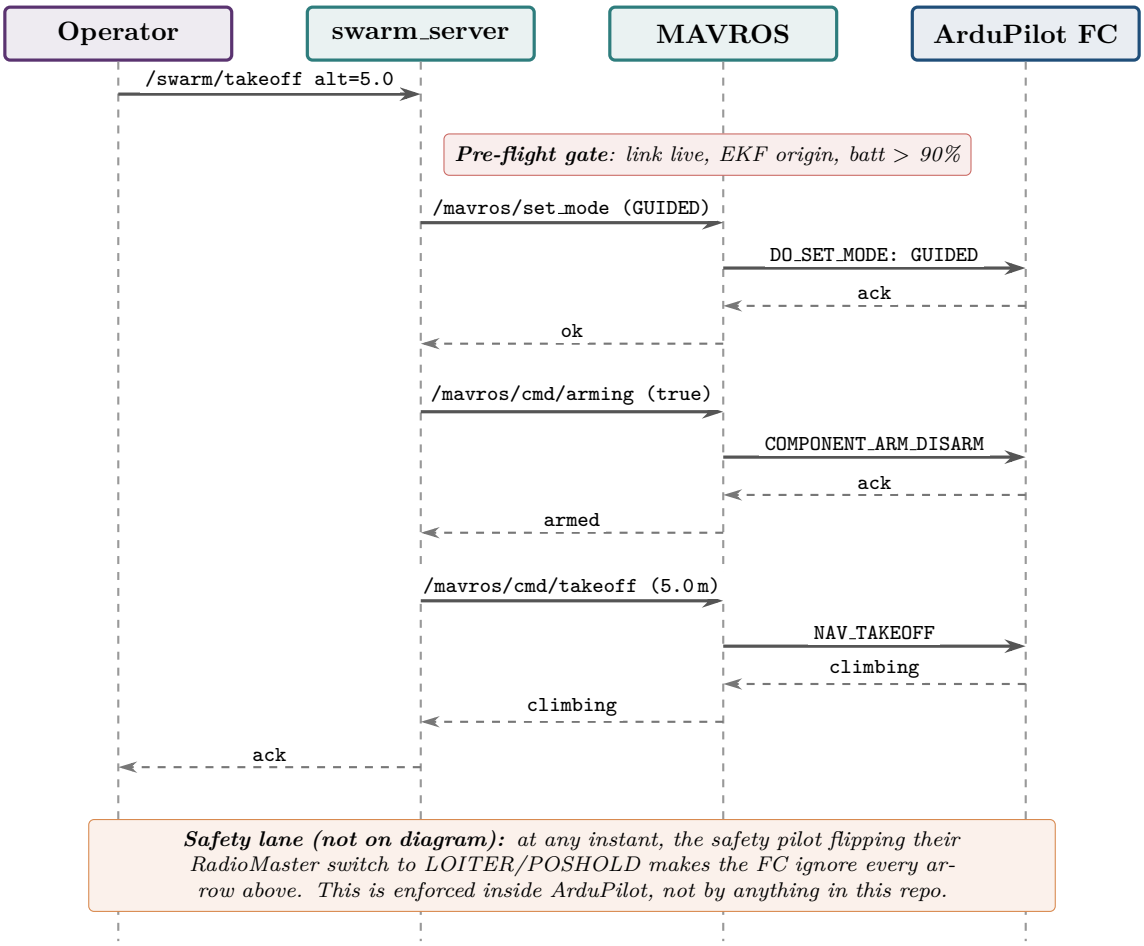
UML sequence diagram of the per-airframe parameter load step from the Phase 2.5b checklist (`config/ardupilot_params/README.md`). This runs once per drone, before any flight.




**"Load from file"** populates the parameter grid only — it does *not* push to the FC. The **Write Params** button is what actually transmits. QGroundControl combines the two; Mission Planner does not. Skipping "Write Params" is the most common cause of a drone arming with unchanged defaults after what looked like a clean parameter load.

## 5. Sequence: arm and takeoff via the swarm services

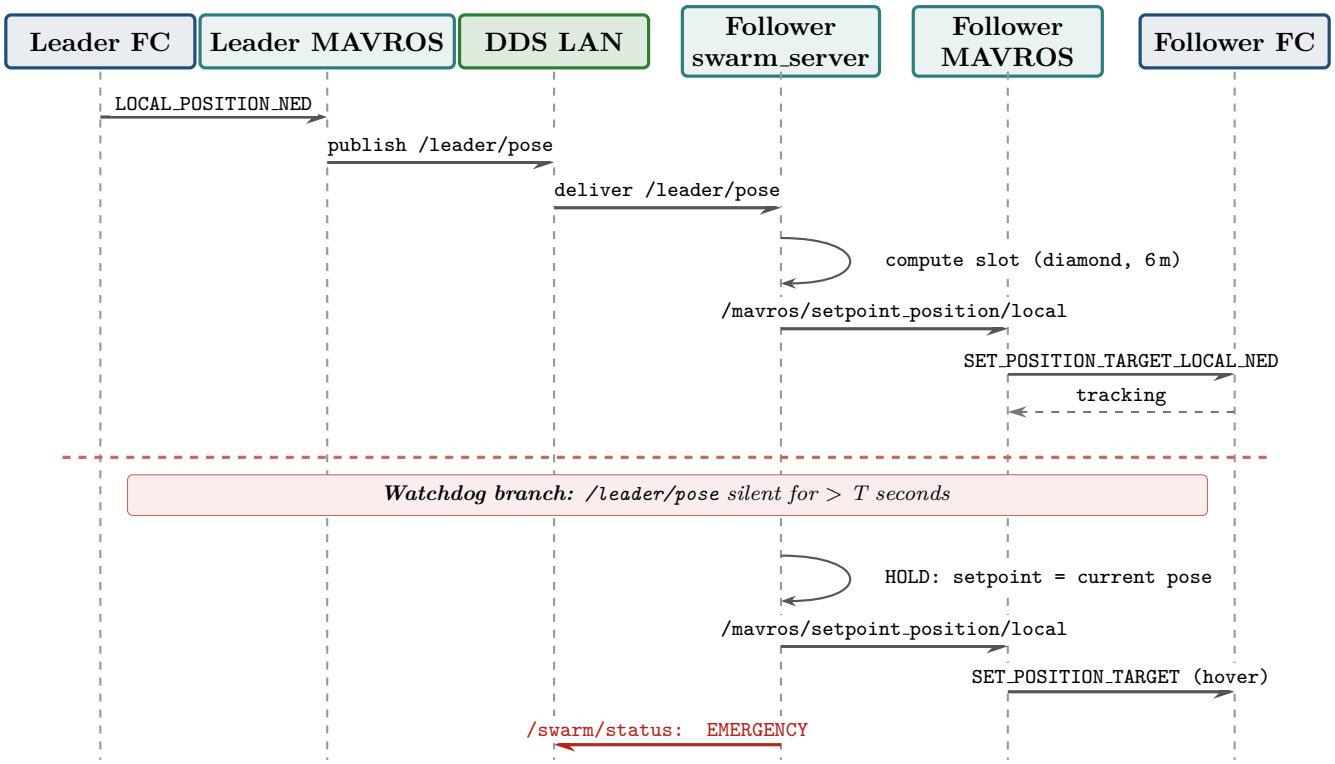
What happens when the operator types `ros2 service call /swarm/takeoff`. Note the parallel safety-pilot lane — their RC switch position is monitored throughout but does not appear as a message; the FC handles RC override internally.




The dim lane on the right is the safety pilot's RC. At any point the safety pilot can flip a mode switch to **LOITER** or **POSHOLD** and the FC ignores all subsequent `swarm_server` commands until they switch back to GUIDED. This is enforced by ArduPilot, not by anything in this repository — it is the layer below all of these arrows.

## 6. Sequence: leader-follow, with watchdog

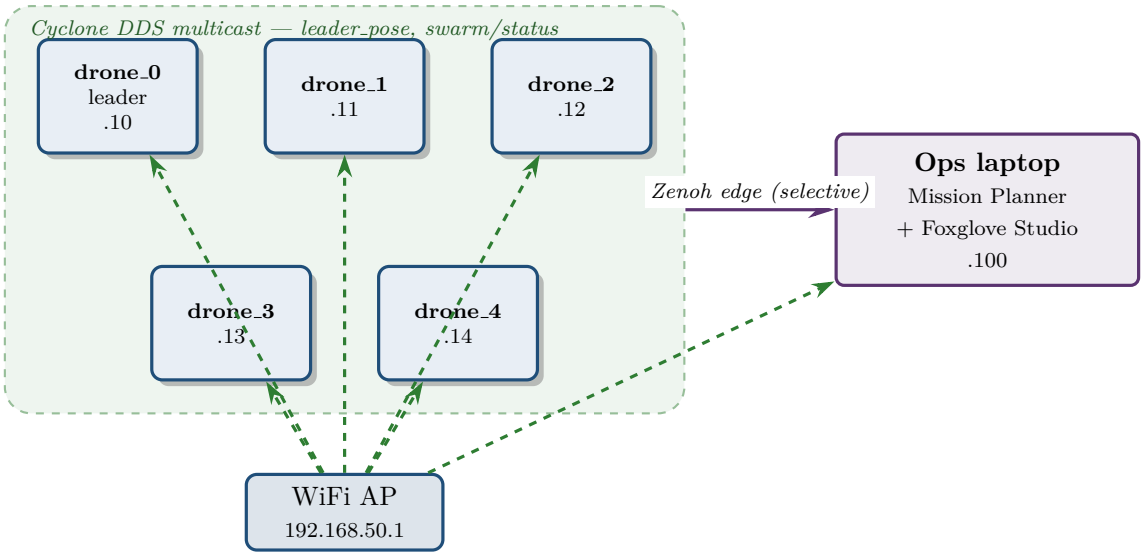
The defining behaviour of Phase 2.5b. Leader pose flows over the Cyclone DDS LAN; each follower's `swarm_server` computes a slot offset and feeds setpoints into MAVROS. A watchdog guards the leader pose against link drop.




**Acceptance gates** (from `PLAN.md` §D, Phase 2.5b): mean horizontal formation error $< 2$ m per follower over $\geq 60$ s of leader motion; watchdog demonstrably holds on a simulated link drop; coordinated land; recorded as `accept/demo_leaderfollow.mcap`.

## 7. Swarm network topology

Five drones on a shared WiFi LAN. Cyclone DDS provides the inter-drone ROS 2 fabric; Zenoh selectively forwards a small subset of topics out to the operator (ADR 0004).

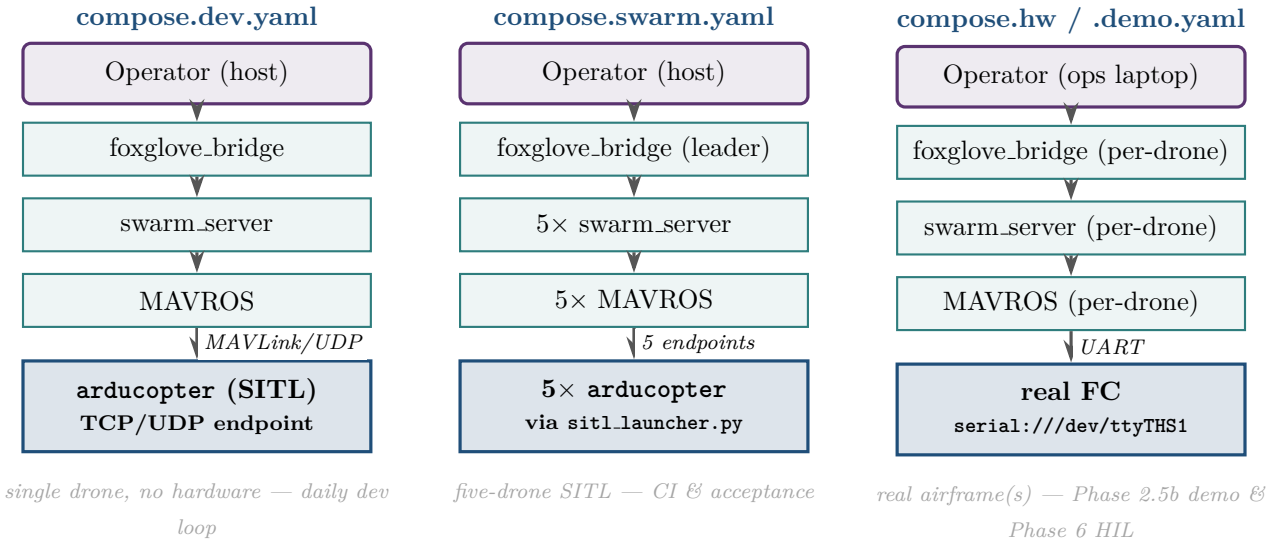



| Address | Host | Role |
|---|---|---|
| `192.168.50.10` | `drone_0` | Leader; `SYSID_THISMAV`=1; manually flown on RC; publishes `/leader/pose` |
| `192.168.50.11--14` | `drone_1..4` | Followers; `SYSID_THISMAV`=2..5; autonomous GUIDED |
| `192.168.50.100` | ops laptop | Mission Planner + Foxglove Studio; runs `ros2 service call /swarm/...` |

**Why this split.** Cyclone DDS is great inside the LAN (multicast, low latency, but chatty on discovery). Zenoh on the edge filters the firehose down to just the topics the operator actually needs — formation pose, status, telemetry summary — avoiding the operator laptop being saturated with intra-swarm chatter (`PLAN.md` §A, ADR 0004).

## 8. The three deployment modes, side by side

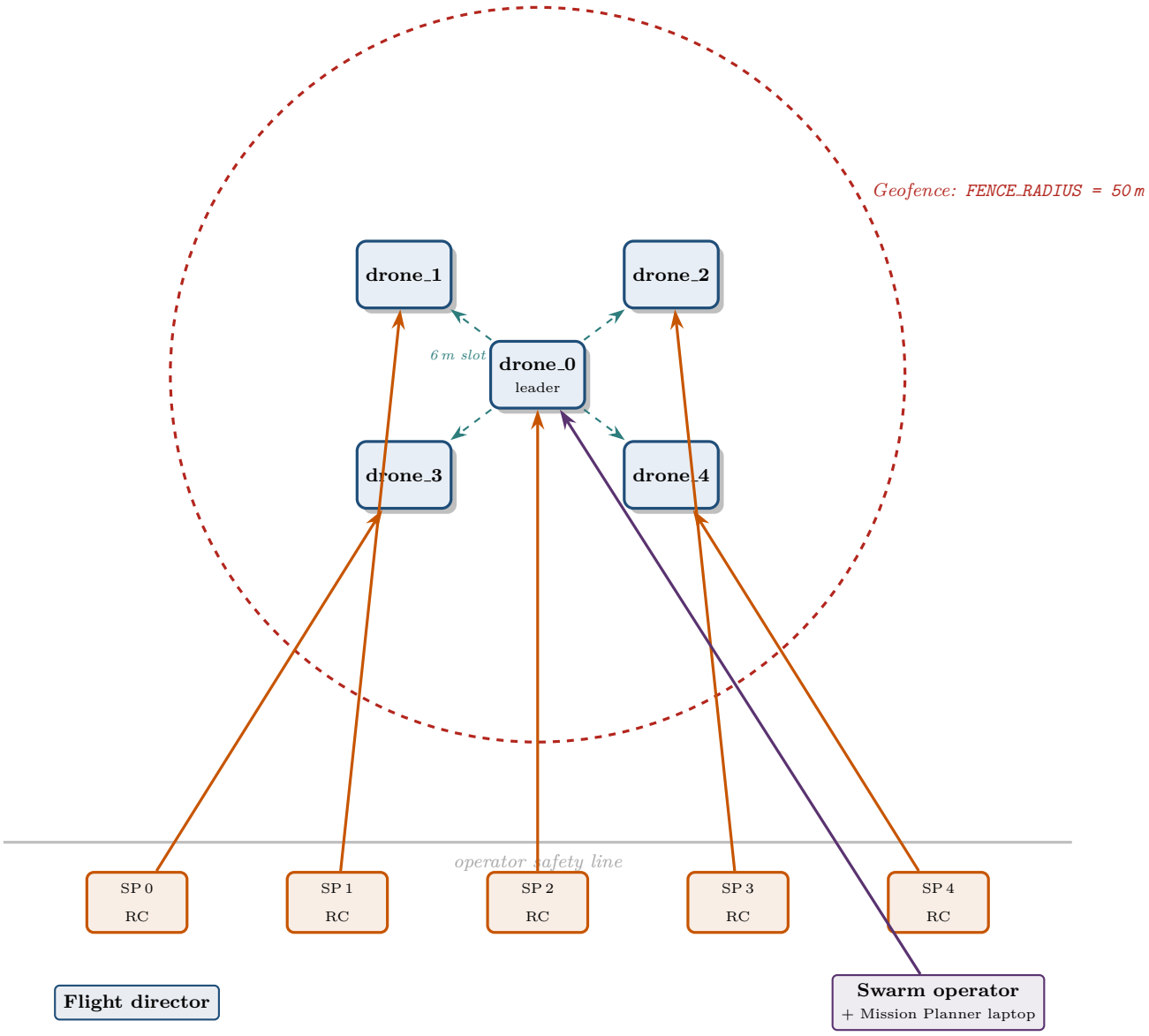
Same architecture, different "FC" box. The MAVROS / swarm_server / Foxglove layer above is identical across all three.



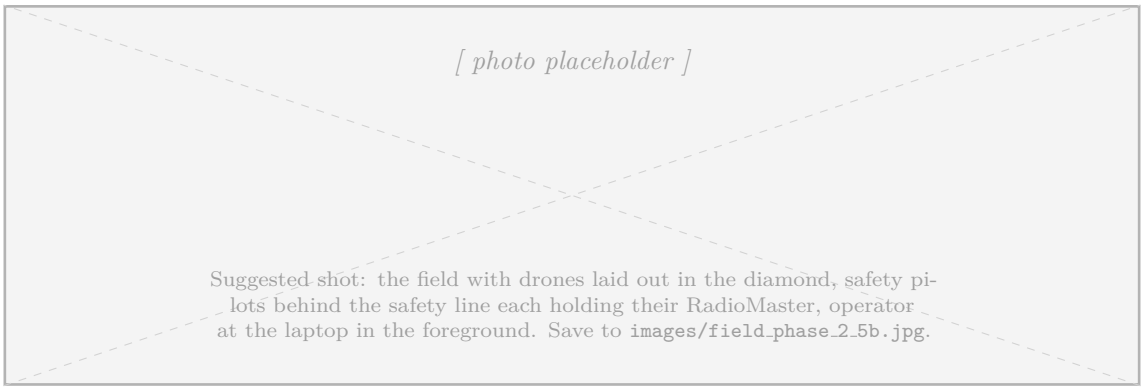

**The portability claim, made concrete.** The MAVROS / swarm_server / foxglove_bridge blocks above the FC box are *identical* across all three columns. They speak MAVLink to whatever happens to be sitting in the FC box — a simulated `arducopter` process, five of them, or your real Pixhawk over a UART. This is why a green `make leaderfollow-smoke` in SITL is a real prerequisite for the bench flight: the same code runs in both.

## 9. Field setup: who stands where (Phase 2.5b)

Top-down view of the operating area during the leader-follow demo. Roles per `docs/runbooks/first_flight.md` §1.



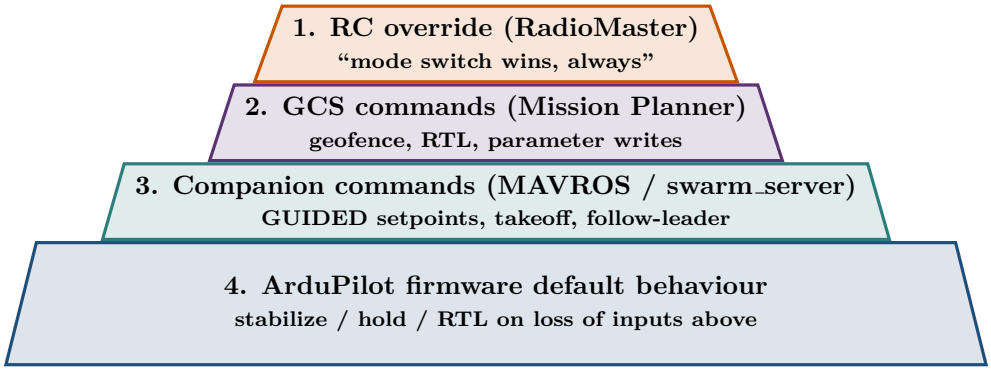

| Role | Count | What they hold / where they look |
|---|---|---|
| Flight director | 1 | No equipment — calls the sequence, owns go/no-go, calls aborts. |
| Safety pilot (SP) | **1 per drone** | Their drone's RadioMaster TX; mode switch ready. |
| Swarm operator | 1 | Ops laptop: Mission Planner (MAVLink monitor) + Foxglove (formation view); runs the `/swarm/*` service calls. |


[ photo placeholder ]
Suggested shot: the field with drones laid out in the diamond, safety pilots behind the safety line each holding their RadioMaster, operator at the laptop in the foreground. Save to images/field_phase_2_5b.jpg.

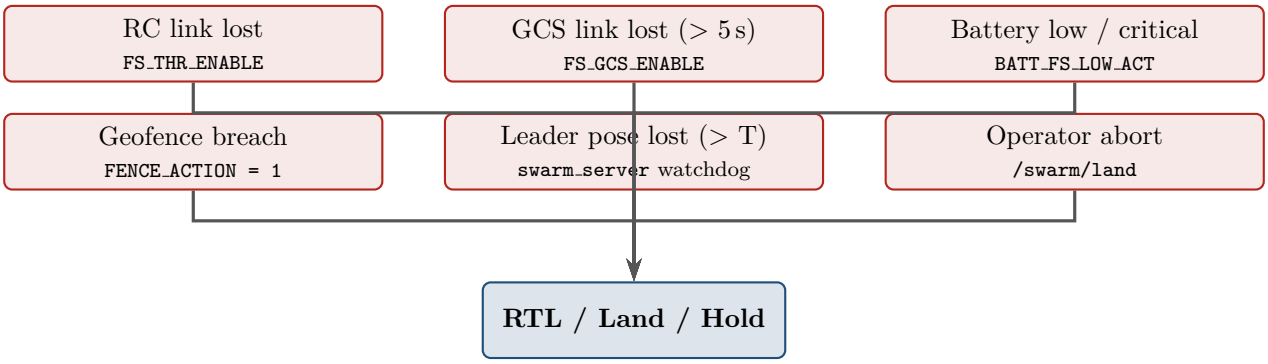
## 10. Authority hierarchy and the failsafe waterfall

Who wins when two channels disagree, and what the firmware does on its own when a channel disappears.

**Authority pyramid** — ArduPilot resolves conflicting commands top-down:




**Failsafe waterfall** — what trips, in what order, when something goes wrong:




**One distinction worth keeping straight**

The first four failsafes are *firmware-level* — they fire inside ArduPilot regardless of the state of the Jetson, MAVROS, or even whether the operator is paying attention. The fifth, *leader pose lost*, is the only swarm-specific failsafe and lives in `swarm_server`; it holds followers in place rather than triggering RTL, because RTL on five drones at once is its own kind of incident.

## 11. Where it all lives in the repo

| What you saw | Section | Where in the repo |
|---|---|---|
| Stack picks | §1 | `PLAN.md` §A (decision record) |
| MAVROS choice | §3 | `docs/adr/0002-mavros-over-ap-dds-for-now.md` |
| DDS + Zenoh split | §3, §7 | `docs/adr/0004-cyclone-dds-plus-zenoh-edge.md` |
| SITL simulator path | §8 | `docs/adr/0003-gazebo-harmonic-ardupilot-gz.md` |
| Leader-follow demo plan | §6, §9 | `docs/adr/0008-leader-follow-demo-integration.md` |
| Single-drone dev compose | §8 | `docker/compose.dev.yaml` |
| Five-drone SITL compose | §8 | `docker/compose.swarm.yaml` |
| Hardware composes (Phase 2.5b) | §8 | `docker/compose.hw.yaml`, `compose.demo.yaml` |
| Param load procedure | §4 | `config/ardupilot_params/README.md` |
| Per-drone parameter files | §4 | `config/ardupilot_params/demo_common.parm`, `drone_N.parm` |
| First-flight checklist | §9, §10 | `docs/runbooks/first_flight.md` |
| Day-to-day operator runbook | §7, §9 | `docs/runbooks/jetson_swarm_operations.md` |
| swarm_server services | §5, §6 | `ros2_ws/src/swarm_server/` |
| Bringup scripts | §9 | `scripts/bringup/demo_swarm.sh` |
| FC link verification | §4 | `scripts/hardware/fc_link_test.py` |
| Phase status (live) | all | `WORKFLOW.md` |

1. `PLAN.md` — the canonical stack picks and roadmap.
2. `docs/architecture/overview.md` — ADR index in suggested reading order.
3. ADRs 0001–0008 — one decision each, with the rationale.
4. `docs/runbooks/first_flight.md` — the hardware demo gate & procedure.

*End of document. To swap a placeholder for an actual photo, drop a JPG into `images/` and uncomment the matching \includegraphics line in the source.*